Posts and Telecommunications Institute of Technology



Information Retrieval
Chapter 8

Faculty: Information Technology 1

Department: Computer Science

Instructor: Pham Hoang Viet

Email: vietph@ptit.edu.vn **Phone:** 0945296388

MAIN CONTENTS

Classification and Machine Learning in IR

8.1 Text classification: Naive Bayes, Vector Space Classification (Rocchio, kNN)

8.2 Learning to Rank: Introducing Pointwise and Pairwise methods

CHAPTER 8 SECTION 1

Text Classification

Naive Bayes, Rocchio and k-Nearest Neighbors

COURSE MAP

Learning outcomes

- Formulate text classification as a supervised-learning problem
- Construct text features using bag-of-words and TF-IDF
- Train and apply a Multinomial Naive Bayes classifier
- Explain conditional independence and Laplace smoothing
- Construct a Rocchio centroid classifier
- Apply kNN using cosine similarity
- Compare probabilistic and vector-space classifiers
- Evaluate classifiers on imbalanced text datasets

d* → *c

Probability, centroids and local neighbors.

Naive Bayes and Vector Space Classification

- Multinomial Naive Bayes
- Rocchio Centroid Classifier
- *k*-Nearest Neighbors
- Training, prediction, and evaluation

Overview of text classification

What Is Text Classification?

- Text classification assigns predefined labels
- Given:
  - A document d
  - A predefined class set: $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$
- Learn a function: $f: \mathcal{D} \rightarrow \mathcal{C}$
- Prediction: $\hat{c} = f(d)$

SUPERVISED DATA

(document, label)

Generalize to unseen documents

Applications

- Spam filtering
- News categorization
- Sentiment analysis
- Topic classification
- Language identification
- Intent detection
- Document routing
- Legal document classification
- Medical coding
- Toxic-content detection

Single-Label and Multi-Label Classification

❖**Single-label classification**

$$y_i \in \mathcal{C}$$

- One document belongs to one class.

❖**Multi-label classification**

$$Y_i \subseteq \mathcal{C}$$

- One document may belong to multiple classes.

❑Example: "AI applications in medical imaging"

Possible labels:

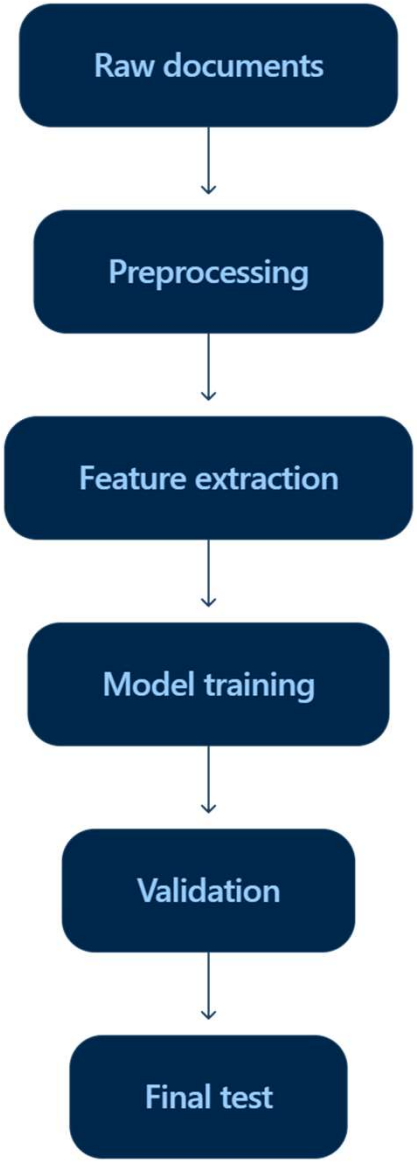
- Artificial intelligence
- Healthcare
- Computer vision

Supervised Learning Setting

- Training set:

$$\mathcal{D}_{\text{train}} = \{(d_1, y_1), (d_2, y_2), \ldots, (d_N, y_N)\}$$

  - where:
  - d_i: document
  - y_i: ground-truth label
- Goal:
  - Learn from labeled documents and generalize to unseen documents



A leakage-free classification pipeline

- **Training set:** estimate model parameters
- **Validation set:** select hyperparameters
- **Test set:** estimate final generalization performance

- Example split:

$$70\%/15\%/15\%$$

- **Important**:

Fit the vocabulary, IDF, and feature-selection method using training data **only**.

Text representation: Counts and TF–IDF serve different models

Bag-of-Words Representation

❖A document is represented by term counts:

$$\vec{x}_d = (x_{1d}, x_{2d}, \ldots, x_{Vd})$$

- where:
  x_{td} = number of occurrences of term t in d

❖Properties:

- Ignores word order
- Produces high-dimensional vectors
- Usually very sparse

Binary representation

$$x_{td} = \begin{cases} 1, & t \text{ occurs in } d \\ 0, & \text{otherwise} \end{cases}$$

❖Focus:

- Whether a term appears
- Not how many times it appears

❖Often used with:

- Bernoulli Naive Bayes
- Short documents
- Presence-sensitive tasks

TF-IDF Representation

$$\text{tf} - \text{idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

$$\text{idf}(t) = \log \frac{N}{\text{df}(t)}$$

❖ Common sublinear TF:

$$\text{tf}(t, d) = 1 + \log f_{t,d}$$

❖ Useful for:

- Rocchio
- kNN
- Linear classifiers

Document Normalization

❖ Length normalization:

$$\hat{\vec{x}}_d = \frac{\vec{x}_d}{\| \vec{x}_d \|_2}$$

❖ Cosine similarity:

$$\text{sim}(d_i, d_j) = \frac{\vec{x}_i \cdot \vec{x}_j}{\| \vec{x}_i \|_2 \| \vec{x}_j \|_2}$$

❖ If vectors are normalized:

$$\text{sim}(d_i, d_j) = \hat{\vec{x}}_i \cdot \hat{\vec{x}}_j$$

Preprocessing Choices

❖ Tokenization

❖ Lowercasing

❖ Stop-word removal

❖ Stemming

❖ Lemmatization

❖ Handling negation

❖ Phrase extraction

❖ Character n-grams

❖ Word n-grams

❖ Rare-term filtering

❑ The best choices depend on the task and language.

Multinomial Naive Bayes

Bayes rule turns classification into likelihood

Probabilistic Classification

❖Predict the class with the highest posterior probability:

$$\hat{c} = \arg\max_{c \in C} P(c \mid d)$$

❖Using Bayes' theorem:

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

P(d) is identical across classes and can be removed from the comparison.

❖Since $P(d)$is identical for all classes:

$$\hat{c} = \arg\max_{c} P(d \mid c)P(c)$$

Why Is It Called "Naive"?

❖Let document d contain features: $x_1, x_2, \ldots, x_V$

❖Naive conditional-independence assumption:

$$P(x_1, \ldots, x_V \mid c) = \prod_{t=1}^{V} P(x_t \mid c)$$

❖Interpretation: Features are conditionally independent given the class.

Variants of Naive Bayes

Multinomial Naive Bayes

❖Uses term frequencies

❖Models token generation

❖Common for topic classification

Multinomial Naive Bayes

❖Uses term frequencies

❖Models token generation

❖Common for topic classification

Gaussian Naive Bayes

❖Models continuous features

❖Usually not the natural choice for word counts

Multinomial Naive Bayes Model

❖Class prior:

$$P(c)$$

❖Class-conditional term probability:

$$P(t \mid c)$$

❖For a document represented by counts x_{td}:

$$P(d \mid c) \propto \prod_{t \in V} P\,(t \mid c)^{x_{td}}$$

❖Classification:

$$\hat{c} = \arg\max_{c} P(c) \prod_{t \in V} P\,(t \mid c)^{x_{td}}$$

Estimating Class Priors

❖Maximum-likelihood estimate:

$$\hat{P}(c) = \frac{N_c}{N}$$

where:

- N_c: number of training documents in class c
- N: total number of training documents

❖Uniform prior alternative:

$$P(c) = \frac{1}{|\mathcal{C}|}$$

Estimating Term Probabilities

❖Without smoothing:

$$\hat{P}(t \mid c) = \frac{T_{ct}}{\sum_{t' \in V} T_{ct'}}$$

where:

- T_{ct}: count of term t in class c
- Denominator: total token count in class c

The Zero-Probability Problem

❖If a term never occurs in class c:

$$T_{ct} = 0 \Rightarrow P(t \mid c) = 0$$

Then:

$$P(d \mid c) = 0$$

even if all other terms strongly support class c.

❖Solution:

Additive smoothing

Additive smoothing prevents zero likelihoods

Laplace Smoothing

❖Add-one smoothing:

$$\hat{P}(t \mid c) = \frac{T_{ct} + 1}{\sum_{t' \in V} T_{ct'} + |V|}$$

❖More generally, add-α:

$$\hat{P}(t \mid c) = \frac{T_{ct} + \alpha}{\sum_{t'} T_{ct'} + \alpha |V|}$$

where: $\alpha > 0$

Unseen terms receive small non-zero probability.

Log scores avoid numerical underflow

❖Direct multiplication:

$$P(c)\prod_{t} P\left(t \mid c\right)^{x_{td}}$$

may underflow numerically.

❖Use logarithms:

$$\text{score}(c,d) = \log P\left(c\right) + \sum_{t} x_{td} \log P\left(t \mid c\right)$$

Multiplication of tiny probabilities becomes stable addition.

❖Prediction:

$$\hat{c} = \arg\max_{c} \text{score}(c,d)$$

Multinomial NB Training Algorithm

For each class c:

1. Count training documents N_c
2. Estimate:

$$P(c) = \frac{N_c}{N}$$

3. Count each term:

$$T_{ct}$$

4. Calculate smoothed probabilities:

$$P(t \mid c) = \frac{T_{ct} + \alpha}{\sum_{t'} T_{ct'} + \alpha \mid V \mid}$$

❖Store log probabilities.

Prediction Algorithm

for each class c:
    score[c] = log P(c)

    for each term t in document d:
        score[c] += count(t, d) × log P(t | c)

return class with the highest score

❖Complexity for a sparse document:
$O(|\mathcal{C}| \times |\text{terms}(d)|)$

❑ Naive Bayes Example

❖Classes:

$$\mathcal{C} = \{\text{Sports}, \text{Technology}\}$$

❖Training token counts:

Term	Sports	Technology
team	8	1
goal	6	0
computer	0	7
software	1	6

❖Total tokens:

$$T_{\text{Sports}} = 15, T_{\text{Technology}} = 14$$

❖Vocabulary size:

$$|V| = 4$$

Smoothing

❖For Sports:

$$P(\text{goal} \mid S) = \frac{6+1}{15+4} = \frac{7}{19}$$

$$P(\text{software} \mid S) = \frac{1+1}{15+4} = \frac{2}{19}$$

❖For Technology:

$$P(\text{goal} \mid T) = \frac{0+1}{14+4} = \frac{1}{18}$$

$$P(\text{software} \mid T) = \frac{6+1}{14+4} = \frac{7}{18}$$

Prediction

❖New document: goal software

- Assume equal priors:

$$P(S) = P(T) = 0.5$$

❖Sports score:

$$P(S)P(\text{goal} \mid S)P(\text{software} \mid S)$$
$$= 0.5 \times \frac{7}{19} \times \frac{2}{19}$$

❖Technology score:

$$P(T)P(\text{goal} \mid T)P(\text{software} \mid T)$$
$$= 0.5 \times \frac{1}{18} \times \frac{7}{18}$$

❖Prediction:

$$\hat{c} = \text{Sports}$$

Bernoulli Naive Bayes

- Binary feature: $B_{td} \in \{0, 1\}$
- Document likelihood: $P(d \mid c) = \prod_{t \in V} P\,(B_t = 1 \mid c)^{B_{td}}[1 - P(B_t = 1 \mid c)]^{1-B_{td}}$
- It uses evidence from: Present terms & Absent terms

Aspect	Multinomial NB	Bernoulli NB
Feature	Term count	Binary presence
Repeated occurrence	Matters	Ignored
Absent terms	Ignored directly	Explicitly modeled
Typical use	Longer documents, topics	Short text, presence patterns
Natural input	Count vectors	Binary vectors

Strengths and Limitations of Naive Bayes

Strengths

- ❖Very fast training and prediction
- ❖Memory efficient
- ❖Works well with sparse high-dimensional data
- ❖Effective with relatively little training data
- ❖Produces class scores with probabilistic interpretation

Limitations

- ❖Unrealistic independence assumption
- ❖Repeated correlated features may be overcounted
- ❖Probability estimates may be poorly calibrated
- ❖Ignores word order in bag-of-words form

Vector Space Classification

Vector Space Classification

- Represent every document as a vector:

$$\vec{d} \in \mathbb{R}^{|V|}$$

- Classification is based on geometric relationships:
  - Distance to a class centroid
  - Similarity to labeled neighbors
  - Direction in the TF-IDF space
- Methods:
  - Rocchio classifier
  - k-Nearest Neighbors

Rocchio Classifier

From Relevance Feedback to Classification

- In relevance feedback, Rocchio modifies a query using document centroids.
- In classification, each class is represented by a centroid:

$$\vec{\mu}_c = \frac{1}{|\ D_c\ |} \sum_{\vec{d} \in D_c} \vec{d}$$

where D_c contains training documents of class c.

- Prediction:

$$\hat{c} = \arg\max_c \operatorname{sim}\left(\vec{d}, \vec{\mu}_c\right)$$

A new document joins the class whose representative direction is closest.

Rocchio Decision Rule

❖**Constructing Class Centroids**

- For class c:

$$\vec{\mu}_c = \frac{1}{N_c} \sum_{i:y_i=c} \vec{x}_i$$

- Recommended steps:
  1. Construct training TF-IDF vectors
  2. Normalize document vectors
  3. Average vectors within each class
  4. Optionally normalize each centroid

$$\hat{\vec{\mu}}_c = \frac{\vec{\mu}_c}{\| \vec{\mu}_c \|_2}$$

❖Using cosine similarity:

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \frac{\vec{x} \cdot \vec{\mu}_c}{\| \vec{x} \| \| \vec{\mu}_c \|}$$

❖Using Euclidean distance:

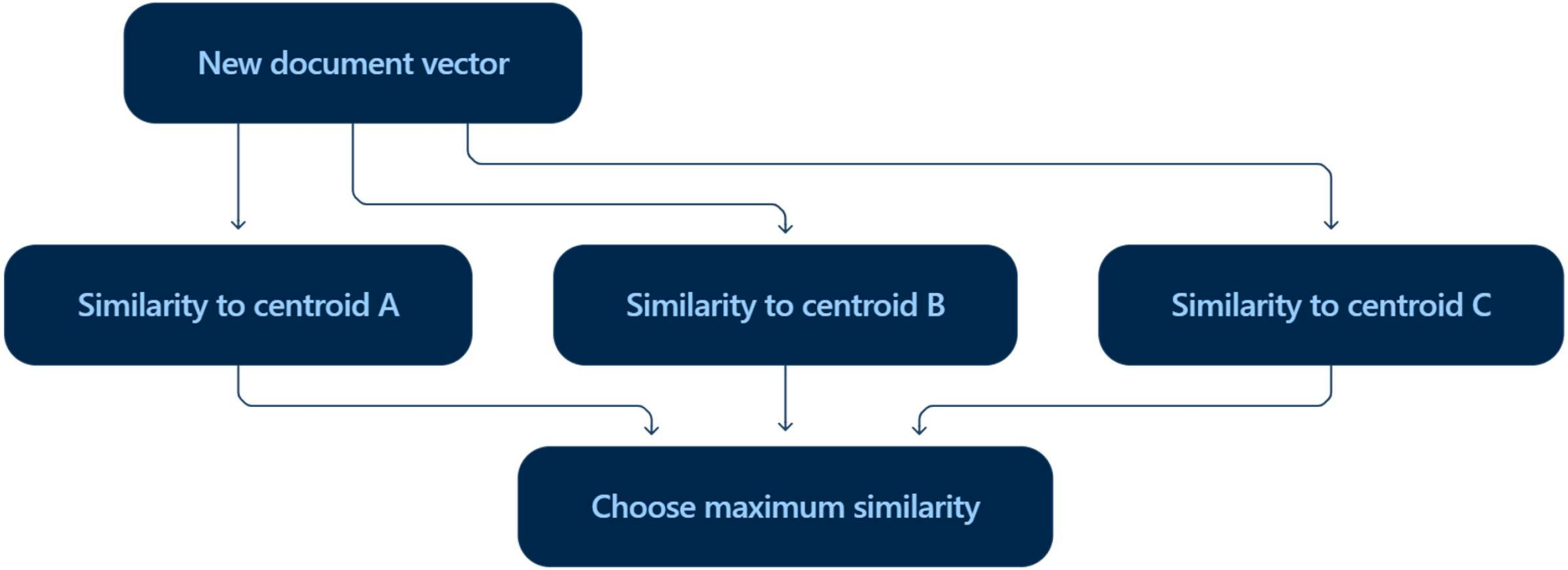
$$\hat{c} = \arg\min_{c \in \mathcal{C}} \| \vec{x} - \vec{\mu}_c \|_2$$

- For unit-normalized vectors:

$$\| \vec{x} - \vec{\mu}_c \|_2^2 = 2 - 2\cos(\vec{x}, \vec{\mu}_c)$$

Geometric Intuition

❖Each class is summarized by one representative point.



❑ Rocchio Example

- Two classes:

$$D_A = \{(1,1),(1,3)\}$$
$$D_B = \{(4,1),(6,1)\}$$

- Centroids:

$$\vec{\mu}_A = (1,2)$$
$$\vec{\mu}_B = (5,1)$$

- New document:

$$\vec{x} = (2,2)$$

- Euclidean distances:

$$\| \vec{x} - \vec{\mu}_A \| = 1$$
$$\| \vec{x} - \vec{\mu}_B \| = \sqrt{10}$$

- Therefore:

$$\hat{c} = A$$

Rocchio Training and Prediction

❖**Training**

for each class c:

centroid[c] = average of its training vectors

normalize centroid[c]

❖**Prediction**

for each class c:

score[c] = cosine(document, centroid[c])

return argmax(score)

❖**Complexity**

- Training: $O(\text{nnz}(X))$
- Prediction: approximately $O(KV)$, or less with sparse vectors

❖**Decision Boundaries**

- For two normalized centroids $\vec{\mu}_1, \vec{\mu}_2$, the boundary satisfies:

$$\vec{x} \cdot \vec{\mu}_1 = \vec{x} \cdot \vec{\mu}_2$$

or:

$$\vec{x} \cdot (\vec{\mu}_1 - \vec{\mu}_2) = 0$$

- Thus, Rocchio produces a linear decision boundary.

Strengths and Limitations of Rocchio Classification

Strengths

- ❖Simple and interpretable
- ❖Very fast training
- ❖Fast prediction
- ❖Low storage cost
- ❖Naturally compatible with TF-IDF
- ❖Suitable for many classes
- ❖Centroid terms explain the class topic
- ❖Strong baseline for topic classification

Limitations

- ❖Assumes each class has a meaningful centroid
- ❖Cannot model complex non-convex class shapes
- ❖A class may contain multiple subtopics
- ❖Sensitive to outliers
- ❖Overlapping centroids cause ambiguity
- ❖Minority subclusters may disappear in the average
- ❖Does not naturally estimate probabilities

k-Nearest Neighbors

For a new document $\vec{x}$:

1. Compute similarity to training documents
2. Select the k nearest neighbors:

$$N_k(\vec{x})$$

3. Predict using neighbor labels

❖Unweighted voting:

$$\hat{c} = \arg\max_c \sum_{\vec{x}_i \in N_k(\vec{x})} I(y_i = c)$$

❖ **Similarity for Text kNN**

- TF-IDF cosine similarity:

$$\text{sim}(\vec{x}, \vec{x}_i) = \frac{\vec{x} \cdot \vec{x}_i}{\| \vec{x} \| \| \vec{x}_i \|}$$

- Cosine distance:

$$d_{\cos}(\vec{x}, \vec{x}_i) = 1 - \text{sim}(\vec{x}, \vec{x}_i)$$

- For normalized vectors:

$$\text{sim}(\vec{x}, \vec{x}_i) = \vec{x} \cdot \vec{x}_i$$

Choosing k: using validation or cross-validation.

Small *k*

- ❖Flexible local decision
- ❖Sensitive to noise and outliers
- ❖High variance

Large *k*

- ❖More stable
- ❖Smoother decision boundary
- ❖May include unrelated neighbors
- ❖May favor majority classes

Weighted kNN

❖Closer neighbors should contribute more:

$$\text{score}(c, \vec{x}) = \sum_{\vec{x}_i \in N_k(\vec{x})} w_i \, I(y_i = c)$$

❖Possible weights:

$$w_i = \text{sim}(\vec{x}, \vec{x}_i)$$

or:

$$w_i = \frac{1}{d(\vec{x}, \vec{x}_i) + \epsilon}$$

❖Prediction:

$$\hat{c} = \arg\max_c \text{score}(c, \vec{x})$$

❑ kNN Example

❖For a new document, the five nearest neighbors are:

Neighbor	Label	Cosine
d_1	Sports	0.91
d_2	Technology	0.87
d_3	Technology	0.82
d_4	Sports	0.61
d_5	Sports	0.58

❖Unweighted vote:

$$\text{Sports} = 3, \text{Technology} = 2$$

❖Prediction:

$$\hat{c} = \text{Sports}$$

❖**Weighted Voting**

- Sports score:
  $0.91 + 0.61 + 0.58 = 2.10$
- Technology score:
  $0.87 + 0.82 = 1.69$
- Prediction remains:
  $\hat{c} = \text{Sports}$
- But the confidence gap becomes visible:
  $2.10 - 1.69 = 0.41$

Handling Class Imbalance in kNN

Possible strategies:

- ❖Distance-weighted voting
- ❖Class-prior correction
- ❖Class-specific voting weights
- ❖Balanced neighbor selection
- ❖Different k_c by class
- ❖Resampling the training set

Prior-corrected vote:

$$\text{score}'(c, x) = \frac{\text{score}(c, x)}{P(c)^{\rho}}$$

Efficient Text kNN

Possible acceleration techniques:

- ❖Sparse matrix multiplication
- ❖Inverted-index candidate generation
- ❖Dimensionality reduction
- ❖Feature selection
- ❖Approximate nearest neighbors
- ❖Locality-sensitive hashing
- ❖Clustering and centroid pruning
- ❖GPU batch similarity

Strengths and Limitations of kNN

Strengths

- ❖Simple and intuitive
- ❖Almost no explicit training cost
- ❖Naturally supports nonlinear boundaries
- ❖Adapts to local class structure
- ❖Can model multimodal classes
- ❖Easy to update with new documents
- ❖Neighbor documents provide explanations

Limitations

- ❖High prediction cost
- ❖Stores all training documents
- ❖Sensitive to k, metric, and feature weighting
- ❖Noise affects local decisions
- ❖Class imbalance affects voting
- ❖Similarity may be weak for short texts
- ❖Naive search scales poorly

Prediction cost:

$$O\left(N_{\text{train}} \times \text{similarity cost}\right)$$

Comparison & Evaluation

Naive Bayes vs. Rocchio vs. kNN

Aspect	Naive Bayes	Rocchio	kNN
Main view	Probabilistic	Class centroid	Local neighbors
Typical features	Term counts	TF-IDF	TF-IDF
Training cost	Low	Low	Minimal
Prediction cost	Low	Very low	High
Storage	Class-term statistics	One centroid per class	All training vectors
Boundary	Often linear in log space	Linear	Nonlinear
Multimodal classes	Limited	Weak	Stronger
Interpretability	Term likelihoods	Centroid terms	Neighbor examples

Parametric and Non-Parametric Views

Naive Bayes

- Fixed number of parameters after vocabulary is fixed
- Learns class and term probabilities

Rocchio

- Stores one representative vector per class
- Strong compression of training dat

kNN

- Non-parametric
- Model size grows with training data
- Complexity retained until prediction time

Global vs. Local Classification

Global models

- ❖Naive Bayes
- ❖Rocchio
- ❖Learn one global representation per class

Local model

- ❖kNN
- ❖Makes decisions using nearby examples

Implication:

- ❖Global methods are efficient
- ❖Local methods can model heterogeneous classes

Which Method Should We Choose?

Choose **Naive Bayes** when:

- ❖ A fast probabilistic baseline is needed
- ❖ Training data is limited
- ❖ Count features are natural

Choose **Rocchio** when:

- ❖ Prediction speed is critical
- ❖ Classes are topically coherent
- ❖ Interpretability matters

Choose **kNN** when:

- ❖ Local examples are informative
- ❖ Classes contain multiple subtopics
- ❖ Prediction latency is acceptable

Model evaluation

Confusion Matrix

❖For binary classification:

	Predicted positive	Predicted negative
Actual positive	TP	FN
Actual negative	FP	TN

❖The confusion matrix is the basis for:

- Precision
- Recall
- F-measure
- Specificity
- Accuracy

Evaluation Protocol

A reliable experiment should:

❖Use stratified data splitting

❖Fit preprocessing only on training data

❖Tune hyperparameters on validation data

❖Evaluate once on the held-out test set

❖Report macro-F1 and micro-F1

❖Inspect the confusion matrix

❖Analyze per-class performance

❖Compare training time, prediction time, and memory

❖**Classification Metrics**

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2\frac{PR}{P + R}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

❖**Macro averaging**

Calculate a metric for each class, then average:

$$F_{1,\text{macro}} = \frac{1}{K}\sum_{c=1}^{K} F_{1,c}$$

❖**Micro averaging**

Aggregate TP, FP, and FN across classes:

$$F_{1,\text{micro}} = \frac{2TP_{\text{all}}}{2TP_{\text{all}} + FP_{\text{all}} + FN_{\text{all}}}$$

Hyperparameters

Naive Bayes

- Smoothing parameter α
- Multinomial vs. Bernoulli
- Feature representation
- Vocabulary filtering

Rocchio

- TF-IDF variant
- Vector normalization
- Distance or similarity measure
- One or multiple centroids per class

kNN

- Number of neighbors k
- Similarity measure
- Weighted or unweighted voting
- Class-imbalance correction

Implementation and practice

Scikit-learn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

model = Pipeline([
    ("tfidf", TfidfVectorizer(
        ngram_range=(1, 2),
        min_df=2
    )),
    ("classifier", MultinomialNB(alpha=1.0))
])

model.fit(train_texts, train_labels)
predictions = model.predict(test_texts)
```

Rocchio Implementation

```
import numpy as np
from sklearn.preprocessing import normalize

class RocchioClassifier:
    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.centroids_ = np.vstack([ X[y == c].mean(axis=0) for c in self.classes_])
        self.centroids_ = normalize(self.centroids_)
        return self

    def predict(self, X):
        X = normalize(X)
        scores = X @ self.centroids_.T
        indices = np.asarray(scores.argmax(axis=1)).ravel()
        return self.classes_[indices]
```

kNN Implementation

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", KNeighborsClassifier(
        n_neighbors=5,
        weights="distance",
        metric="cosine",
        algorithm="brute"
    ))
])

model.fit(train_texts, train_labels)
predictions = model.predict(test_texts)
```

Suggested Experimental Study

❖Compare:

- Count + Multinomial NB
- Binary features + Bernoulli NB
- TF-IDF + Rocchio
- TF-IDF + unweighted kNN
- TF-IDF + weighted kNN

❖Test:

$$\alpha \in \{0.01, 0.1, 0.5, 1, 2\}$$
$$k \in \{1, 3, 5, 10, 20, 50\}$$

❖Report:

- Macro-F1
- Micro-F1
- Training time
- Prediction time
- Model size

Common Implementation Mistakes

- Computing IDF using the test set
- Selecting vocabulary before data splitting
- Applying dense conversion to a huge sparse matrix
- Using Euclidean distance on unnormalized counts
- Forgetting smoothing in Naive Bayes
- Using k larger than the training set
- Reporting only accuracy on imbalanced data
- Tuning parameters on test results
- Ignoring duplicated documents across splits

❑ Worked Discussion

❖Suppose a dataset has:

- 1,000,000 training documents
- 100 classes
- Strict real-time latency
- Mostly coherent topic classes
- Limited prediction memory

❖Which classifier is initially most appropriate?

A. Naive Bayes
B. Rocchio
C. kNN
D. All are equally suitable

Summary - Key Takeaways

- Text classification assigns predefined labels to documents.
- Naive Bayes models class priors and class-conditional term probabilities.
- Smoothing prevents zero probabilities.
- Log probabilities prevent numerical underflow.
- Rocchio represents each class using a centroid.
- kNN predicts from local labeled neighbors.
- Rocchio is compact and fast.
- kNN is flexible but expensive at prediction time.
- Feature representation is as important as the classifier.
- Evaluation must consider class imbalance, latency, and memory.

❑ Discussion Questions

1. Why can Naive Bayes work despite its unrealistic assumption?
2. Why is TF-IDF natural for Rocchio and kNN?
3. When does a single centroid poorly represent a class?
4. Why is kNN called a lazy learner?
5. How does class imbalance affect the three methods?
6. Which method provides the clearest example-based explanation?
7. Can Rocchio be extended to multiple centroids per class?
8. Why may accuracy be misleading?

❑ Programming exercises

Build a text-classification system on:

- 20 Newsgroups
- Reuters-21578
- AG News
- BBC News
- A Vietnamese news dataset

Tasks:

1. Create a leakage-free pipeline
2. Implement Naive Bayes, Rocchio, and kNN
3. Tune hyperparameters using validation data
4. Report macro-F1 and micro-F1
5. Compare runtime and memory
6. Analyze the confusion matrix
7. Explain five misclassified documents
8. Identify the best model under different deployment constraints

CHAPTER 8 SECTION 1

Learning to Rank

Pointwise learning and pairwise Ranking SVM

COURSE MAP

Learning outcomes

- Formulate ranking as a supervised-learning problem
- Construct query–document feature vectors
- Distinguish pointwise, pairwise, and listwise approaches
- Apply pointwise regression and classification
- Convert relevance labels into pairwise preferences
- Explain the Ranking SVM objective
- Construct training pairs for Ranking SVM
- Train and evaluate a ranking model
- Identify common data-leakage and sampling problems

$$q,d \rightarrow s$$

Learn the score; evaluate the order.

Pointwise and Pairwise Approaches

- Learning ranking functions from labeled data
- Pointwise regression and classification
- Pairwise preference learning
- Ranking SVM

Introduction

Why Do We Need Learning to Rank?

❖ **Traditional ranking**

$$s(q,d) = \mathrm{BM25}(q,d)$$

❖ **Learning to Rank**

$$s(q,d) = f_\theta\big(\phi(q,d)\big)$$

where:

- q: query
- d: document
- $\phi(q,d)$: query–document feature vector
- f_θ: learned scoring function

❖ A modern retrieval system may combine many heterogeneous signals:

- BM25 score
- Query likelihood
- Title matching
- Phrase matching
- Semantic similarity
- PageRank or authority
- Document freshness
- Click signals
- Document quality
- User or context features

❖ Challenge: How should these signals be combined?

- Manual weights are difficult to tune
- Relevance judgments provide supervision
- The learned score reranks candidate documents

Ranking as Supervised Learning

❖Training data:

$$\mathcal{D} = \{(q_i, d_{ij}, y_{ij})\}$$

where:

- q_i: query i
- d_{ij}: candidate document j for query i
- y_{ij}: relevance label

❑Example relevance grades:

$$y_{ij} \in \{0, 1, 2, 3, 4\}$$

❖**Ranking Is Query-Grouped**

- For query q_1:

$$\{d_{11}, d_{12}, d_{13}, \dots\}$$

- For query q_2:

$$\{d_{21}, d_{22}, d_{23}, \dots\}$$

- Ranking is performed within each query group:

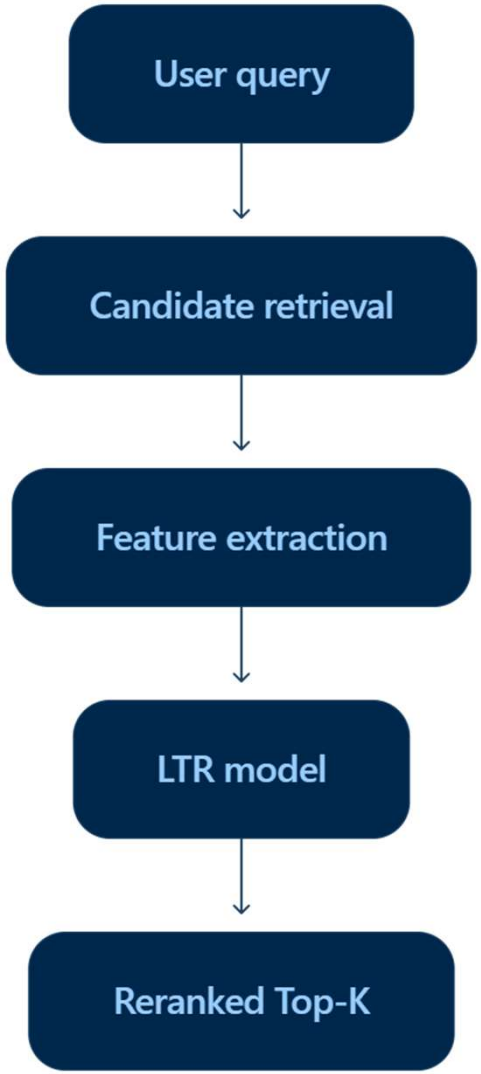
$$d_{ij} \succ d_{ik} \iff s(q_i, d_{ij}) > s(q_i, d_{ik})$$

- Documents from different queries are not directly compared.

Two-Stage Retrieval Architecture

Typical pipeline:

- ❖BM25 retrieves Top-K candidates
- ❖Features are computed for candidates
- ❖LTR reranks the candidate set



Query–Document Features

$$\phi(q, d) = [x_1, x_2, \ldots, x_M]$$

- Example:

Feature	Value
BM25 body score	8.42
BM25 title score	4.71
Query-term coverage	0.80
Phrase match	1
Semantic similarity	0.76
Document freshness	0.65
Authority score	0.83

Types of Ranking Features

Query-dependent features

- BM25
- TF-IDF cosine
- Query likelihood
- Exact phrase match
- Query-term coverage
- Embedding similarity

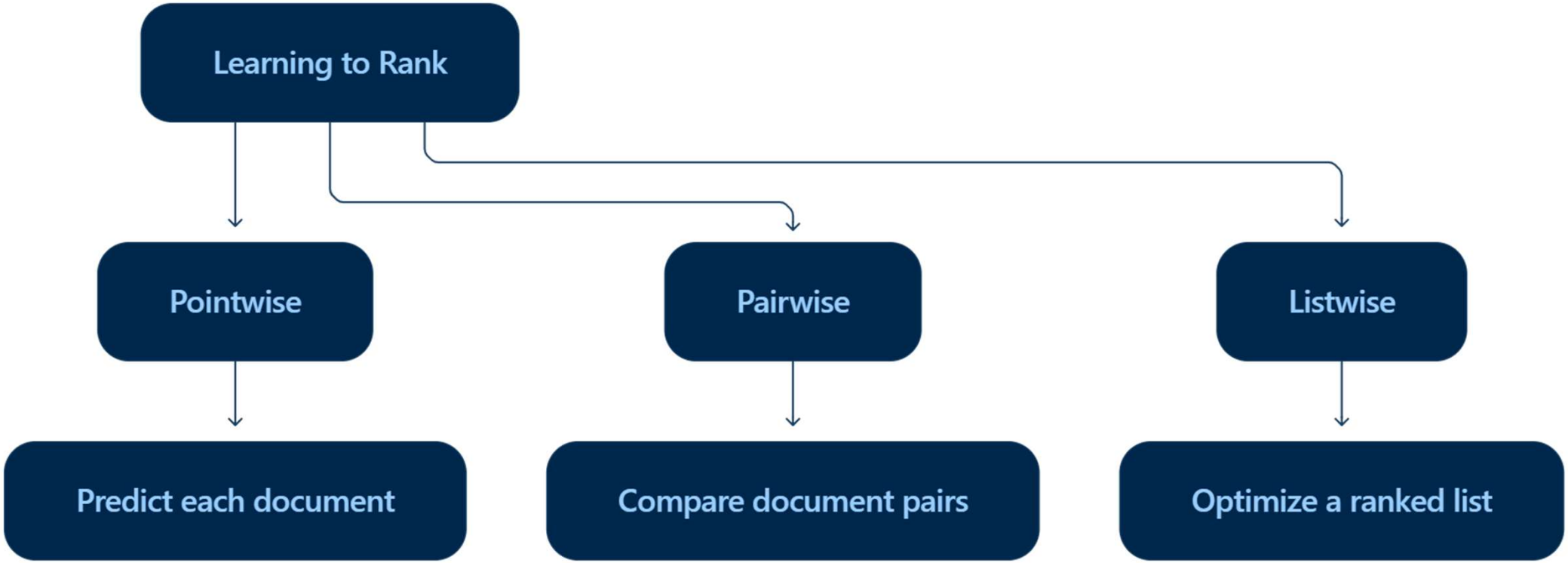
Query-independent features

- PageRank
- Document quality
- Popularity
- Freshness
- Spam score
- Source authority

Query features

- Query length
- Query category
- Query clarity
- Navigational vs. informational intent

Three LTR Paradigms



Pointwise Approach

❖Training instance:

$$(\phi(q,d), y)$$

❖Learn:

$$f_\theta\big(\phi(q,d)\big) \approx y$$

❖At prediction time:

- Score each document independently
- Sort documents by predicted score

❖Possible formulations:

- Regression
- Binary classification
- Multiclass or ordinal classification

Pairwise Approach

❖Training instance:

$$\left(\phi(q, d_i), \phi\left(q, d_j\right), d_i \succ d_j\right)$$

❖Preference:

$$d_i \succ d_j \iff y_i > y_j$$

❖Learn:

$$s(q, d_i) > s\left(q, d_j\right)$$

❖The model focuses on avoiding incorrectly ordered pairs.

Listwise Approach

- Training instance:

$$(q, \{d_1, d_2, \dots, d_n\}, \{y_1, y_2, \dots, y_n\})$$

- The entire ranked list is considered jointly.
- Examples:
  - ListNet
  - ListMLE
  - LambdaMART
  - Differentiable NDCG objectives

Comparison of LTR Paradigms

Aspect	Pointwise	Pairwise	Listwise
Learning unit	One document	Document pair	Ranked list
Target	Relevance label	Preference	List ordering
Main goal	Predict relevance	Order pairs correctly	Optimize list quality
Complexity	Low	Moderate	High
Ranking awareness	Limited	Direct	Strongest
Example	Regression	Ranking SVM	ListNet

Pointwise Learning to Rank

Pointwise regression predicts relevance

- Given relevance grade:

$$y_{qd} \in \{0, 1, 2, 3, 4\}$$

- Learn a regression function:

$$\hat{y}_{qd} = f_\theta\big(\phi(q, d)\big)$$

- Prediction:

$$s(q, d) = \hat{y}_{qd}$$

- Rank documents in descending order of $\hat{y}_{qd}$.

A standard regression loss learns labels, then sorting creates the ranking.

Linear Pointwise Regression

❖Scoring function:

$$s(q,d) = \vec{w}^{\top}\phi(q,d) + b$$

❖Squared-error objective:

$$\min_{\vec{w},b} \frac{1}{2} \| \vec{w} \|^2 + C \sum_{q,d} \left(y_{qd} - s(q,d) \right)^2$$

❖Interpretation:

- $\vec{w}$: feature weights
- b: intercept
- C: fit–regularization trade-off

❑ Pointwise Regression Example

- Suppose:

$$\phi(q, d) = [\text{BM25}, \text{title match}, \text{freshness}]$$

- Learned function:

$$s(q, d) = 0.6x_1 + 1.2x_2 + 0.3x_3$$

- Documents:

Document	BM25	Title	Freshness	Score
d_1	4	1	0.5	3.75
d_2	5	0	0.8	3.24
d_3	2	1	0.9	2.67

- Ranking:

$$d_1 \succ d_2 \succ d_3$$

Pointwise Binary Classification

❖Convert labels into:

$$y_{qd} \in \{0, 1\}$$

❖Learn:

$$P\big(y = 1 \mid \phi(q, d)\big)$$

❖Ranking score:

$$s(q, d) = P\big(y = 1 \mid \phi(q, d)\big)$$

❖Possible models:

- Logistic regression
- Naive Bayes
- Decision tree
- Neural network

Pointwise Multiclass Classification

- Treat relevance grades as classes:

$$y_{qd} \in \{0, 1, 2, 3, 4\}$$

- Estimate:

$$P\big(y = r \mid \phi(q, d)\big)$$

- Expected relevance score:

$$s(q, d) = \sum_{r=0}^{R} r\, P\big(y = r \mid \phi(q, d)\big)$$

Ordinal Nature of Relevance

- Relevance labels are ordered:

$$0 < 1 < 2 < 3 < 4$$

- But ordinary multiclass classification treats classes as nominal.
- Consequences:
  - Predicting 4 instead of 3 is a small error
  - Predicting 0 instead of 4 is a large error
  - Standard classification loss may treat both equally
- Possible solution: Ordinal regression

Limitations of Pointwise Methods

- Treat query–document instances independently
- Do not directly optimize relative order
- Ignore document competition within a query
- Regression errors do not always imply ranking errors
- Label scale may be arbitrary
- Large classes or queries may dominate training
- Standard losses differ from MAP and NDCG

Pairwise Learning to Rank

Labels create pairwise preferences

❖For the same query q:

$$y_i > y_j \Rightarrow d_i \succ d_j \Rightarrow s(q, d_i) \succ \mathrm{s}(\mathrm{q}, d_j)$$

Equal-label documents provide no preference.

❖A pairwise training example can be represented as:

$$\left(\phi(q, d_i), \phi\left(q, d_j\right), +1\right)$$

❖Reverse pair:

$$\left(\phi\left(q, d_j\right), \phi(q, d_i), -1\right)$$

Feature differences turn ranking into classification

❖Define:

$$\vec{z}_{ij} = \phi(q, d_i) - \phi(q, d_j)$$

A positive projection means document i should rank above document j.

❖Pair label:

$$r_{ij} = \begin{cases} +1, & d_i \succ d_j \\ -1, & d_j \succ d_i \end{cases}$$

❖Train a binary classifier:

$$r_{ij} \approx \text{sign}(\vec{w}^{\top} \vec{z}_{ij})$$

❖ **Why Does the Difference Work?**

- Linear scoring function:
  $$s(q, d) = \vec{w}^{\top} \phi(q, d)$$
- Preference condition:
  $$s(q, d_i) > s(q, d_j)$$
- Therefore:
  $$\vec{w}^{\top} \phi(q, d_i) > \vec{w}^{\top} \phi(q, d_j)$$
  which implies:
  $$\vec{w}^{\top} [\phi(q, d_i) - \phi(q, d_j)] > 0$$

❑ Pair Generation Example

- For one query:

Document	Relevance
d_1	3
d_2	2
d_3	0
d_4	0

❖Valid preferences:

$$d_1 \succ d_2$$
$$d_1 \succ d_3, d_1 \succ d_4$$
$$d_2 \succ d_3, d_2 \succ d_4$$

❖No preference between:

d_3 and d_4

Pairwise Classification Error

❖A pair is correctly ordered if:

$$(y_i - y_j)[s(q, d_i) - s(q, d_j)] > 0$$

❖Pairwise error:

$$L_{\text{pair}} = \sum_{q} \sum_{i<j} I[(y_i - y_j)(s_i - s_j) \leq 0]$$

❖Goal:

Minimize the number of incorrectly ordered pairs.

Advantages and Limitations of Pairwise Learning

Advantages

- ❖Directly models relative preference
- ❖Less sensitive to the absolute label scale
- ❖Naturally reflects ranking order
- ❖Uses comparisons within the same query
- ❖Can work with preference judgments
- ❖Converts ranking into binary classification

Limitations

- ❖Number of pairs may be quadratic
- ❖Pairs are not independent
- ❖Queries with many candidates generate more pairs
- ❖All pair errors may be treated equally
- ❖Does not directly optimize top-heavy metrics
- ❖Pairwise preferences may be inconsistent
- ❖Pair sampling can bias the model

❑ Example

For one query:

Document	BM25	Title match	Freshness	Label
d_1	8.0	1	0.6	3
d_2	9.0	0	0.8	1
d_3	5.0	1	0.9	2
d_4	7.0	0	0.4	0

- **Pointwise Transformation**
  - Pointwise training instances:

$$([8.0, 1, 0.6], 3)$$
$$([9.0, 0, 0.8], 1)$$
$$([5.0, 1, 0.9], 2)$$
$$([7.0, 0, 0.4], 0)$$

  - The model predicts relevance grades independently.

- **Pairwise Transformation**
  - Preferences:

$$d_1 \succ d_2, d_1 \succ d_3, d_1 \succ d_4$$
$$d_3 \succ d_2, d_3 \succ d_4$$
$$d_2 \succ d_4$$

  - Example difference:

$$\vec{z}_{13} = \vec{x}_1 - \vec{x}_3 = [3.0, 0, -0.3]$$

  - Constraint:

$$3w_1 - 0.3w_3 \geq 1 - \xi_{13}$$

❑ Different Loss Behaviors

- True labels:

$$y_1 = 3, y_2 = 2$$

- **Prediction A**

$$\hat{y}_1 = 10, \hat{y}_2 = 9$$

  - Large regression error
  - Correct ranking

- **Prediction B**

$$\hat{y}_1 = 2.4, \hat{y}_2 = 2.5$$

  - Small regression error
  - Incorrect ranking

Ranking SVM

From SVM to Ranking SVM

❖Binary SVM learns:

$$y_i(\vec{w}^\top \vec{x}_i + b) \geq 1 - \xi_i$$

❖Ranking SVM learns:

$$\vec{w}^\top [\phi(q, d_i) - \phi(q, d_j)] \geq 1 - \xi_{ij}$$

for every preference:

$$d_i \succ d_j$$

Ranking SVM Scoring Function

❖Linear ranking function:

$$s(q, d) = \vec{w}^\top \phi(q, d)$$

❖For a preferred pair:

$$d_i \succ d_j$$

desired constraint:

$$s(q, d_i) \geq s(q, d_j) + 1 - \xi_{ij}$$

Equivalently:

$$\vec{w}^\top (\vec{x}_i - \vec{x}_j) \geq 1 - \xi_{ij}$$

Ranking SVM Objective

$$\min_{\vec{w},\xi} \frac{1}{2} \| \vec{w} \|^2 + C \sum_{(i,j)\in\mathcal{P}} \xi_{ij}$$

$$\text{subject to } \vec{w}^\top \left(\vec{x}_i - \vec{x}_j \right) \geq 1 - \xi_{ij},$$

$$\xi_{ij} \geq 0$$

where $\mathcal{P}$ is the set of preference pairs.

Hinge-Loss Form

❖Constraint form is equivalent to:

$$\min_{\vec{w}} \frac{1}{2} \| \vec{w} \|^2 + C \sum_{(i,j)\in\mathcal{P}} \max\left(0, 1 - \vec{w}^\top (\vec{x}_i - \vec{x}_j)\right)$$

❖Pairwise hinge loss:

$$\ell_{ij} = \max\left(0, 1 - (s_i - s_j)\right)$$

Geometric Interpretation

❖Each preference pair becomes a difference vector:

$$\vec{z}_{ij} = \vec{x}_i - \vec{x}_j$$

❖Ranking SVM seeks:

$$\vec{w}^\top \vec{z}_{ij} > 0$$

❖Reverse pair:

$$\vec{z}_{ji} = -\vec{z}_{ij}$$

❖Thus, pairwise ranking becomes a maximum-margin classification problem.

Why Is There Usually No Intercept?

❖Suppose:

$$s(q,d) = \vec{w}^{\mathsf{T}}\vec{x} + b$$

Then:

$$s(q,d_i) - s(q,d_j) = (\vec{w}^{\mathsf{T}}\vec{x}_i + b) - (\vec{w}^{\mathsf{T}}\vec{x}_j + b)$$

❖Therefore:

$$s_i - s_j = \vec{w}^{\mathsf{T}}(\vec{x}_i - \vec{x}_j)$$

❖The intercept b cancels.

❑ Ranking SVM Example

❖Features:

$$\phi(q, d) = [\text{BM25, title match}]$$

❖Documents:

$$\vec{x}_A = (4, 1), y_A = 2$$
$$\vec{x}_B = (5, 0), y_B = 0$$

❖Since:

$$A \succ B$$

❖Difference vector:

$$\vec{z}_{AB} = \vec{x}_A - \vec{x}_B = (-1, 1)$$

❖Constraint:

$$-w_1 + w_2 \geq 1 - \xi$$

Ranking SVM Prediction

❖After training:

$$\vec{w} = [w_1, w_2, \ldots, w_M]$$

❖For a new query q:

1. Generate candidate documents
2. Compute $\phi(q, d)$
3. Calculate:

$$s(q, d) = \vec{w}^{\top} \phi(q, d)$$

4. Sort candidates by decreasing score

Pair construction is needed only during training.

Feature Scaling

❖Features may have very different ranges:

Feature	Possible range
BM25	0–30
Cosine similarity	0–1
PageRank	10^{-8}–10^{-2}
Document length	10–100,000
Phrase match	0or 1

❖Possible transformations:

- Standardization
- Min–max scaling
- Log transformation
- Query-level normalization

Query-Level Feature Normalization

For feature m within query q:

❖**Min–max normalization**

$$x'_{qdm} = \frac{x_{qdm} - \min_{d'} x_{qd'm}}{\max_{d'} x_{qd'm} - \min_{d'} x_{qd'm}}$$

❖**Z-score normalization**

$$x'_{qdm} = \frac{x_{qdm} - \mu_{qm}}{\sigma_{qm}}$$

Build training pairs

Number of Possible Pairs

❖For n_q documents of query q:

- maximum pairs $= \frac{n_q(n_q-1)}{2}$
- Example:

$$n_q = 1{,}000 \Rightarrow 499{,}500 \text{ pairs}$$

❖Across many queries, exhaustive pair generation may be infeasible.

Which Pairs Should Be Generated?

❖Generate pairs only when:

- Documents belong to the same query
- Their relevance labels differ:

$$y_i \neq y_j$$

❖Common strategies:

- All unequal-label pairs
- Relevant vs. non-relevant pairs
- Adjacent relevance-grade pairs
- Random pair sampling
- Hard-negative sampling
- Top-position-focused pairs

Pair Sampling Trade-offs

❖**Uniform random sampling**

- Simple
- May sample many easy pairs

❖**Hard-pair sampling**

- Focuses on current errors
- More informative
- May overfocus on noisy examples

❖**Grade-gap sampling**

- Weight by:

$$| y_i - y_j |$$

- Emphasizes clear preference differences

❖**Rank-sensitive sampling**

- Prioritizes pairs affecting top results

Balancing Pair Directions

❖If all pairs are stored as:

$$\vec{x}_{\text{better}} - \vec{x}_{\text{worse}}$$

with label $+1$, a standard classifier sees only one class.

❖Possible solutions:

1. Add reverse examples:

$$\vec{x}_{\text{worse}} - \vec{x}_{\text{better}}, -1$$

2. Alternate pair orientation and assign ± 1
3. Use a ranking-specific optimizer directly

Query Weighting

- Queries may generate very different numbers of pairs.
- Without correction:
  queries with many documents dominate the loss
- Query-normalized pair weight:

$$w_{qij} = \frac{1}{|\mathcal{P}_q|}$$

- Weighted objective:

$$\min_{\vec{w}} \frac{1}{2} \| \vec{w} \|^2 + C \sum_q \sum_{(i,j) \in \mathcal{P}_q} w_{qij} \ell_{qij}$$

Evaluating Learning to Rank

Ranking Evaluation Metrics

❖Common metrics:

- Precision@k
- Recall@k
- Mean Average Precision
- Mean Reciprocal Rank
- NDCG@k

❖For graded relevance:

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

❖**Pairwise Accuracy**

$$\text{PairAcc} = \frac{\text{number of correctly ordered preference pairs}}{\text{total number of preference pairs}}$$

❖A pair $(i, j)$is correct if:

$$(y_i - y_j)(s_i - s_j) > 0$$

❖Limitation:

- High pairwise accuracy does not guarantee high NDCG@10.

Why Ranking Metrics Differ from Pairwise Loss

- Pairwise loss:
  - Treats pair inversions similarly
  - Does not explicitly model rank position
- NDCG:
  - Gives larger weight to top ranks
  - Uses graded relevance gains
  - Depends on the full list
- Consequently: lower pairwise loss does not imply higher NDCG

Data Splitting by Query

- Correct split:

$$Q_{\text{train}} \cap Q_{\text{validation}} = \emptyset$$
$$Q_{\text{train}} \cap Q_{\text{test}} = \emptyset$$

- All documents for one query must remain in the same split.
- Incorrect:
  - Randomly splitting query–document rows.

Position and Click Bias

❖Observed clicks depend on:

$$P(\text{click}) = P(\text{examination}) \times P(\text{attractiveness or relevance})$$

❖Higher-ranked documents receive more exposure.

❖Problems:

- Position bias
- Presentation bias
- Selection bias
- Trust bias

❖Possible correction:

- Randomization experiments
- Propensity weighting
- Click models
- Human relevance judgments

Implementing Ranking SVM

Pair Construction Pseudocode

```
for each query q:
    documents = documents_for_query(q)

    for each pair (di, dj):
        if relevance(di) == relevance(dj):
            continue

        if relevance(di) > relevance(dj):
            z = features(di) - features(dj)
            label = +1
        else:
            z = features(dj) - features(di)
            label = +1

        store or sample pair z
```

Training with a Linear SVM

```
from sklearn.svm import LinearSVC
ranker = LinearSVC(C=1.0, fit_intercept=False)
ranker.fit(pair_vectors, pair_labels)
```

Prediction:

```
scores = document_features @ ranker.coef_.ravel()
ranking = scores.argsort()[::-1]
```

Safe Pair Generation

```
def make_pairs(X, y, query_ids):
    pair_X, pair_y = [], []

    for qid in unique(query_ids):
        idx = indices_for_query(qid)

        for i, j in sampled_pairs(idx):
            if y[i] == y[j]:
                continue

            sign = 1 if y[i] > y[j] else -1
            z = X[i] - X[j]

            pair_X.append(z)
            pair_y.append(sign)

            pair_X.append(-z)
            pair_y.append(-sign)

    return pair_X, pair_y
```

Ranking SVM Hyperparameters

- Important choices:
  - Regularization parameter C
  - Feature scaling
  - Pair-sampling strategy
  - Pair weights
  - Grade-gap threshold
  - Number of pairs per query
  - Linear vs. kernel model
- Typical search:

$$C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$$

- Select using validation NDCG, not training hinge loss.

Linear and Kernel Ranking SVM

❖Linear Ranking SVM

$$s(q,d) = \vec{w}^{\top}\phi(q,d)$$

- Efficient
- Interpretable
- Suitable for large datasets

❖Kernel Ranking SVM

$$K\left(\vec{z}_{ij}, \vec{z}_{kl}\right)$$

- Models nonlinear relations
- Higher training and prediction cost
- Difficult to scale to many pairs

Comparison of Pointwise and Ranking SVM

Aspect	Pointwise	Ranking SVM
Learning example	Query–document	Document pair
Target	Relevance grade	Relative preference
Loss	Regression/classification	Pairwise hinge
Ranking awareness	Indirect	Direct
Data size	Number of documents	Number of pairs
Label-scale sensitivity	Higher	Lower
Training complexity	Lower	Higher
Prediction	Score and sort	Score and sort

Strengths and Limitations of Ranking SVM

Strengths

- ❖Directly learns document preferences
- ❖Convex optimization objective
- ❖Global optimum for the linear formulation
- ❖Supports high-dimensional feature vectors
- ❖Less dependent on the absolute relevance scale
- ❖Fast prediction using a linear scoring function
- ❖Interpretable feature weights
- ❖Strong classical LTR baseline

Limitations

- ❖Pair generation may be expensive
- ❖Equal treatment of pairs may conflict with NDCG
- ❖Linear model may miss feature interactions
- ❖Sensitive to feature scaling
- ❖Queries with many pairs may dominate
- ❖Pairwise preferences may be noisy
- ❖Does not directly optimize the final ranking metric

❑ Worked Exercise

- Two documents for the same query:

$$\vec{x}_A = [3, 1, 0.8], y_A = 2$$
$$\vec{x}_B = [5, 0, 0.4], y_B = 0$$

- Feature order: BM25, Title match, Freshness
- Given:

$$\vec{w} = [0.4, 1.5, 0.5]$$

- Tasks:
  - Compute s_A
  - Compute s_B
  - Determine the predicted order
  - Calculate the pairwise hinge loss

- **Solution**
  - Document A:

$$s_A = 0.4(3) + 1.5(1) + 0.5(0.8) = 3.1$$

  - Document B:

$$s_B = 0.4(5) + 1.5(0) + 0.5(0.4) = 2.2$$

  - Predicted order:

$$A \succ B$$

  - Margin:

$$s_A - s_B = 0.9$$

  - Hinge loss:

$$\ell_{AB} = \max(0, 1 - 0.9) = 0.1$$

❑ Pair Construction Exercise

- For one query:

Document	Label
d_1	2
d_2	2
d_3	1
d_4	0

- Questions:
  - How many unequal-label pairs exist?
  - List all preference relations.
  - Which pair should not be generated?
  - Which pairs have the largest grade gap?

- **Solution**
  - Valid preferences:

$$d_1 \succ d_3, d_1 \succ d_4$$
$$d_2 \succ d_3, d_2 \succ d_4$$
$$d_3 \succ d_4$$

  - Number of valid preference pairs: 5
  - No preference:

$$d_1 \text{ vs. } d_2$$

  - Largest grade-gap pairs:

$$(d_1, d_4), (d_2, d_4)$$

Summary - Key Takeaways

- Learning to Rank learns a scoring function from relevance data.
- Documents are ranked within query groups.
- Pointwise methods predict relevance for each document independently.
- Pairwise methods learn relative document preferences.
- Ranking SVM applies maximum-margin learning to feature differences.
- Its objective minimizes regularization plus pairwise hinge loss.
- Pair generation and sampling strongly affect training.
- Query-level splitting is required to avoid leakage.
- Pairwise loss is not identical to MAP or NDCG.
- Final model selection should use ranking metrics.

Common Mistakes

- ❖Creating pairs across different queries
- ❖Randomly splitting query–document rows
- ❖Using equal-label pairs as preferences
- ❖Fitting feature scaling on test data
- ❖Forgetting to balance pair directions
- ❖Generating all pairs without memory control
- ❖Selecting $c$using the test set
- ❖Evaluating only with classification accuracy
- ❖Ignoring candidate-generation recall
- ❖Using click labels without bias correction

❑ Suggested Experimental Configurations

Compare:

1. BM25 baseline
2. Pointwise linear regression
3. Pointwise logistic regression
4. Ranking SVM with all pairs
5. Ranking SVM with sampled pairs
6. Grade-gap-weighted Ranking SVM

❖Tune:

$C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$

❖Evaluate:

- MAP
- NDCG@5
- NDCG@10
- Pairwise accuracy
- Training time
- Prediction time

❑ Discussion Questions

1. Why does pointwise regression not directly optimize ranking?
2. Why should pairs come from the same query?
3. Why does the intercept disappear in Ranking SVM?
4. Should all preference pairs have equal weights?
5. Why can higher pairwise accuracy produce lower NDCG?
6. How should pairs be sampled for large datasets?
7. When might pointwise learning outperform pairwise learning?
8. How can Ranking SVM incorporate query intent?

❑ Programming exercises

Build a two-stage ranking system:

1. Use BM25 to retrieve Top-100 documents
2. Construct query–document features
3. Train a pointwise regression model
4. Train a Ranking SVM
5. Rerank the same candidate sets
6. Evaluate MAP and NDCG@10
7. Compare pairwise accuracy
8. Analyze feature weights
9. Analyze queries improved or harmed by LTR

❖Suggested datasets:

- LETOR 4.0
- MSLR-WEB10K
- MSLR-WEB30K
- Yahoo! Learning to Rank
- TREC collections